Introduction

The software engineering landscape has changed considerably in the last few years due to the rise of generative AI. When we talk about generative AI, we're talking about deep learning models like LLMs (Large Language Models), which are capable of code generation, writing documentation, and artifact design. Tools like Copilot, ChatGPT, and Gemini have been adopted by the industry at an increasing rate, becoming an important tool to the average software engineer. The pace of the adoption of these tools has brought about a difficult conflict between the demands of the industry and the academic objectives of institutions. While the industry increasingly demands AI proficiency for the sake of work efficiency, academia faces a struggle to find ways to integrate AI tools without compromising essential skill acquisition. Educators must find new ways to balance the apparent necessity of familiarizing students with these industry-standard tools against the risk of students being overreliant on them and bypassing the critical stage of struggle known as "deep learning."

In response to this rise of generative AI, many institutions have attempted to establish guidelines to make it easier for students and educators to navigate ethical uses of AI. Take Carleton's "golden rules" for using generative AI, which reminds students that AI does not think for the individual and that it deprives them of the struggle necessary to learn essential skills. Other schools, such as Harvard and Stanford universities, have implemented similar policies to emphasize the importance of not relying on the technology, and guidelines for proper use with an emphasis on writing assignments.

However, these guidelines fail to address the specific educational needs of learning software development, given that completely sidelining generative AI might give students a disadvantage in their careers. It is important to have guidelines that cover the middle gaps in which AI use in the field of software engineering might be beneficial for students, without encouraging reliance on the technology.

Defining Productivity in Software Engineering

To assess the impact of generative AI, it is important to look at more than just an archaic definition of productivity. In the past, productivity was used to measure the rate of outputs in the software engineering industry, like lines of code per unit of time. However, current literature argues that simplistic measure fails at capturing the abstractions of modern software engineering, which is highly collaborative in nature. Furthermore, higher coding speed does not always equate to increased productivity if the code lacks quality or scalability (Aboubakar, 2025).

Coutinho et al (2024) illustrates the complexity of productivity using the SPACE framework (Satisfaction, Performance, Activity, Communication, and Efficiency). The work shows that engineering productivity is not purely dependent on speed, but that instead it relies just as much on engineer satisfaction (removal of tedious tasks) and team communication as it does on raw activity (work performed by engineers as lines of code produced). These concepts serve as an important reminder: Simply generating code faster may actively damage the

productivity of a team if it leads to a degradation of code quality and team communication. Therefore, we cannot assume that just because generative AI is being quickly adopted in the industry constitutes any proof that reliance on the technology is inherently positive. Instead, we must take a closer look at how the quality of the engineering is affected by the use of generative AI.

Pedagogical Impacts of AI in the Learning Process

Classroom AI integration is fundamentally changing the learning process, offering students cognitive relief but also risks of learning reduction. Generative AI tools can significantly reduce the load of students associated with syntax and boiler plate code. Up to 84% of students in a software engineering course used AI to troubleshoot, optimize, and write code (Salomon et al, 2025). Theoretically, this should free up mental space for higher-level code design and time for reasoning. However, this "offloading" can fundamentally change the workflow. Given unrestricted access to AI, students tended to bypass the requirements of design phases and jumped promptly into the implementation phase. While this increases the speed of work, it does so at the risk of taking away from students' critical thinking time for good design.

Another significant concern of allowing the use of generative AI is the bypassing of "deep learning," a process of struggle necessary for productive learning (York, 2023). While AI excels at brainstorming, it creates a risk of putting students in a position in which they generate code that they cannot understand. In this way, students miss the opportunity to develop problem-solving skills and strategies. Furthermore, over-reliance on AI can negatively impact collaborative skills, as reducing direct interpersonal collaboration turns the AI into the developer's only "surrogate" teammate (Salomon et al, 2025).

Finally, the eye for code quality and security cannot be developed efficiently when students rely heavily on generative AI. It was found that while AI tools like Copilot excel at syntax correctness, they tend to struggle with design abstractions, often failing to suggest idiomatic code or to avoid "code smells" (Pudari and Ernst, 2023). Even more, Generative AI models can suffer from "hallucinations" in which they can make up documentation or libraries that don't actually exist, and they can lead students down unproductive or even detrimental paths. Without a proper pedagogical framework to teach students how to decide on the use of AI and its outputs, they risk producing flawed or unsafe software.

Motivating Aspect

The current state of software engineering and its education reveals a lack of structure and pedagogical norms regarding generative AI. Most courses currently only operate at extremes, either allowing the unrestricted use of the technology, which risks endangering skill acquisition, or banning the technology altogether, which leaves the students unprepared for the modern workforce. This project is motivated by the need to fill this gap. Instead of viewing the use of AI in education as a "yes" or "no" binary, this project seeks to develop assignments that motivate

the cultivation of “AI literacy” while urging students to maintain a critical eye for quality, security, and design.

AI Models

To better understand how students actually interact with AI tools in CS257, we decided to compare multiple large language models instead of relying on just one. In reality, students are not using a single “official” AI. They switch between tools depending on availability, familiarity, and perceived quality. Because of this, focusing on only one model would give a narrow and unrealistic picture of how AI is being used in practice. Comparing several models allowed us to observe consistent patterns as well as meaningful differences in how these systems generate code, explain concepts, and support learning.

We selected ChatGPT, Gemini, Claude, DeepSeek, and Grok because they reflect the tools students are most likely to encounter or already use. Some are accessible through personal accounts, while others are available through institutional platforms. At Carleton College specifically, students are encouraged to use large language models through Amplify, which provides access to approved AI tools in a way that protects student data and prevents it from being used for external model training. This matters because it lowers the barrier to responsible AI use while addressing privacy concerns, making AI a realistic and supported part of students’ academic workflow rather than something hidden or unofficial.

To keep the comparison fair, we designed a controlled task setup. Each model was given the same HTML file and the same prompt, so any differences in output would come from the models themselves rather than the instructions. We chose CSS generation as the main task because it is common in CS257 assignments and represents a realistic use case where students might turn to AI. CSS work often involves structure, layout reasoning, and iterative refinement, which makes it a good lens for observing how models balance correctness and usability.

Some models produced very polished and technically complete solutions, but provided a limited explanation of how the code actually worked. For example, ChatGPT often generated the most refined CSS, including well-structured layouts and visually appealing styling, yet the responses sometimes appeared as finished solutions rather than learning-oriented guidance. This makes the tool very efficient for completing tasks, but it can make it harder for students to understand the reasoning behind the layout decisions or how different CSS properties interact.

In contrast, Grok and DeepSeek tended to produce simpler and more readable CSS that was easier to follow step by step. Their outputs often made the structure of the styling clearer, which can support learning, especially for students still developing foundational CSS skills. However, these solutions were sometimes less visually polished or lacked advanced refinements, which means that the result worked correctly but did not always look as professional.
Gemini and Claude generally fell somewhere in between these two extremes. They often provided reasonably clean and functional CSS while also offering some explanation of what the code was doing, which makes their outputs easier for students to revise and build upon.

These observations reinforced the idea that the choice of AI model can directly affect how much students learn from using AI. Without guidance, students may gravitate toward tools that generate full solutions quickly, even if that bypasses meaningful engagement with the material. This comparison helped motivate the need for clear AI-use guidelines in CS257, not to restrict access to tools, but to shape how students interact with them. By understanding what these models do well and where they fall short, we were able to better connect real student behavior with pedagogical goals, which ultimately informed the structure and focus of the guidelines discussed later.

Guidelines

The AI student guidelines that were developed for CS257 emerged directly from patterns and gaps identified in the literature review. As the research highlighted both the rapid integration of AI into education and the risks associated with its unstructured use. It became clear that a structured, pedagogically rooted framework was necessary. Rather than banning AI and risk students losing proficiency in a tool that is rapidly expanding in the market. Or the inverse, unrestricted use of AI, that would potentially harm student growth, the guidelines were designed as a response to these two tensions.

The literature consistently emphasizes the rapid introduction of AI tools into the workforce and the growing expectation that students develop AI proficiency to meet industry needs. This proficiency can range from engaging critically, using AI as a debugging partner, and working with it on problem-solving. However, studies also identified significant risks when AI use is left unchecked. These issues include overreliance on logic generation, solution generation, and unstructured debugging that bypass meaningful engagement with debugging and problem-solving processes. At the same time, productivity research in software engineering further complicates this discussion. Coutinho et al. (2024) describe productivity through the SPACE framework, emphasizing that software engineering success is not determined by speed alone. Instead, it depends on the quality and reliability of produced code, the developer's well-being, the reduction of tedious work, effective communication in teams, and sustained efficiency and workflow. While AI tools may enhance certain dimensions, they also undermine performance and deeper cognitive engagement if relied upon excessively. These concerns motivated the need to define clear boundaries that preserve learning while acknowledging AI's presence.

While the literature offered theoretical discussions of academic integrity, cognitive offloading, and skill development. There was less clarity on how these concerns would translate into a course-level implementation. This gap was clearly visible in programming-heavy courses such as CS257, where AI can influence multiple stages of work. While Carleton does have guidelines that help shape general use cases, our group identified key categories in CS257 where AI use could be better structured. The core areas identified were:

- **Design**

    - This section focuses on using AI to inspire ideas, explore alternative approaches, and refine design decisions. The emphasis is on supporting creativity and critical thinking rather than replacing the student's original design process.
- **Programming**
    - This section addresses the use of AI to assist with understanding concepts, clarifying syntax, and debugging errors. AI should serve as a support tool to enhance comprehension, not as a substitute for writing and reasoning through code independently.
- **Testing**
    - This section centers on using AI to help design and improve test cases, interpret error messages, and suggest debugging strategies. However, students are expected to write and reason through their own test cases first to ensure they understand the underlying logic before consulting AI.
- **Learning**
    - This section emphasizes AI as a learning tool. It may be used to generate explanations of concepts, create practice problems, or assist with information gathering. However, students are strongly encouraged to first attempt to read, analyze, and reason through the material independently before turning to AI.

Rather than issuing broad prohibitions, the guidelines provide concrete examples of acceptable and unacceptable use for each topic. For example, using AI to brainstorm high-level design approaches may be permitted, while generating full program solutions or asking AI to debug entire codebases would fall outside acceptable use. This approach makes expectations explicit while preserving space for productive AI engagement.

While the guidelines do explain proper engagement and how to critically interact with AI. We also identified different AI models that would best suit the needs of students. Students can generally engage with these models through Amplify or through their organization's email on Google Gemini. These models provide students a secure place to engage with AI, so their data is not shared in training these models.

Through the use of these models, we also identified another use case for AI, context training. Context training refers to the practice of providing AI models with relevant course materials, such as assignment specifications, starter code, or project files (see Appendix Figure A1 - A3). So, the responses are grounded in specific constraints and goals of the course. Rather than beginning a generic chat with no background information, students would upload relevant files and establish a defined context before asking questions. For example, a student working on a CS257 project could upload their current codebase and assignment instructions, then ask AI to explain why a specific function is failing or clarify how a design requirement could be interpreted. Similarly, a student preparing for an exam might provide lecture notes and request

targeted practice questions aligned with the terminology of the course. This structured approach helps ensure that AI assistance remains focused and relevant. It reduces the risk of overly generic or misaligned solutions and encourages students to engage more critically with their own materials. Context training promotes a deeper understanding while still leveraging AI as a supportive tool.

The guidelines intentionally position themselves between two extremes: unrestricted AI use, which risks undermining learning, and complete restriction, which ignores the realities of modern computing practice. Instead, AI is framed as a support tool, one that enhances productivity and reflection without replacing core cognitive work. The primary goal of the guidelines is to balance productivity with learning. AI tools can increase efficiency, but without constraints, they may reduce deep engagement with programming concepts. By defining appropriate use cases, the guidelines encourage students to critically engage with AI while maintaining ownership of their learning process. Importantly, the guidelines are intended to be flexible rather than strict, serving as a guide for responsible use rather than a rigid enforcement mechanism.

In this way, the guidelines represent the practical outcome of the literature review. They translate theoretical concerns about cognitive dependency, academic integrity, and skill development into course norms that allow students to critically collaborate with AI. Rather than treating AI as a threat or a solution, the guidelines provide a structured pathway for critically engaging with AI in CS257.

Integration of AI Guidelines

Prior to drafting out our assignments, we had some questions for faculty and students to help guide our planning. For students, we wanted to gauge what type of tools were commonly used, how often they were used, what kind of strategies they used when navigating AI tools, etc. As for faculty, our focus prioritized classroom policy and the general expectations of the software design final project. We interviewed each professor with questions regarding their classroom policy, expectations for the final project with the potential addition of AI, and personal opinion on the usage of large language models.

Based on the experimentation and a survey, we settled on working with three AI tools: ChatGPT, Claude, and Google Gemini. During our experimentation, we narrowed down our selection from various AI models and tools including ChatGPT, Google Gemini, Claude, Deepseek, Grok, and Cursor for code generation of CSS and conceptual learning of Git. While ChatGPT had the most complicated design choice for CSS, we found that Google Gemini and Claude were the best contenders for overall experience and generation of simpler code in a classroom setting. Gemini and Claude were able to generate a CSS file without overcomplications, keeping the file relatively short with little extra color added despite the target design having more color variation. Since the goal of the utilization of these AI tools is collaboration as outlined in our guidelines, we prioritized working with tools that kept the style

sheet basic and easier to understand from a beginner's perspective. With these three tools in mind, we created a survey and out of the students who said "yes" to engaging with AI tools, ChatGPT and Copilot (that utilizes GPT) were the commonly submitted answers for all purposes (coding in class, non-coding conceptual help, personal project, research, and internship). Coming at a close second was Google Gemini followed by Claude.

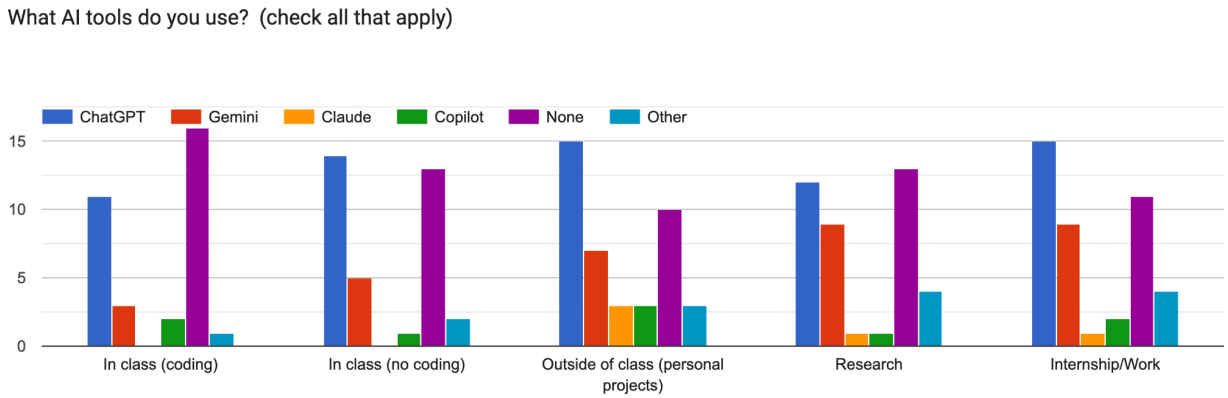

Figure 1. Bar chart displaying the results from a survey asking what AI tools they use (if any) for in class coding, in class no coding, personal projects, research, or internship and work.

Once we established the logistics before building an activity extension or assignment, we moved on to creating four assignments and seven in class activities that followed our Software Design AI Student Guidelines. Assignments cover four large skill building categories including database and SQL management, unit testing, HTML, and CSS. The goal of our guidelines is to help students navigate various AI tools, reduce ambiguity on AI policy, and provide sample prompts for effective AI usage.

In addition to larger assignments we created optional extension in-class exercises with topics and prompts that function as guiding questions and prompts. Unlike longer assignments that require a production and skill building component, these activities are intended to help students learn and understand certain concepts and encourage them to critically think about other aspects related to the class that may be outside of the scope of the project. Each exercise focuses on a skill such as drafting an effective prompt, identifying hallucinations and errors, and asking clarifying questions.

The Database AI assignment prioritizes identifying discrepancies in existing SQL code and fixing issues with the help of AI. Students are tasked with examining the database and first list the problems in the database without the help of AI. After the initial diagnosis, students are then expected to experiment with the AI tool while reflecting on areas where the AI disagreed with the student analysis.

Our Testing AI extension assignment covers unit testing and building a test suite with the help of AI. Students are expected to brainstorm edge cases that the current starter test code does not cover. Rather than feeding in code and letting the generative model produce test cases,

students are expected to fully write their entire test suite, using AI as a guide for ideas rather than a “worker”.

The HTML assignment touches on programming and learning by using AI to generate HTML code. With two options, students can choose between different approaches to getting an AI model to generate code. The first one involves feeding in a paragraph and long description of a desired webpage, and having AI produce the code. Similarly, the second approach involves feeding in individual snippets and working with multiple prompts, making changes when necessary. Both options require the student to reflect and analyze the AI response while also doing external research on tags or topics that may be unfamiliar.

Finally, the CSS assignment utilizes AI to identify potential issues, while at the same time, developing a stylesheet with a target web page image, starter style sheet, and html structure. Similarly to the database assignment, students are first encouraged to find the solution without the help of AI. Then, with a better understanding of the starter code and a stronger foundation, the student can explore more advanced style choices with the help of AI. A common issue that was observed during our experimentation were redundancies in the AI generated CSS code. Through this assignment, students will be able to build their proofreading skills, as a result, be able to notice redundancies and remove them.

Through our survey, faculty interviews, and experimentation with various AI tools, we learned that both students and professors are navigating uncertainty around proper and effective AI usage. Many students are using generative AI tools, but often lack understanding on the best strategies for using them effectively and ethically. Faculty members had differing opinions, but overall expressed both concerns regarding academic integrity and loss of foundational learning. Students and faculty members both recognize AI’s growing role in software development as something that needs to be addressed. These insights helped shape our assignment and class activity design. Rather than allowing students to depend on generated code from AI, we structured activities that position AI as a collaborator or higher level tool for brainstorming. Each assignment requires students to engage with the material independently first, reflecting on the AI outputs after an initial overview. This allows students to analyze the discrepancies between their reasoning and the model’s responses, preventing them from developing bad practices that could be misled by the AI. Overall, our assignments aim to reframe AI not as a threat to software design, but rather as a tool that students can be more comfortable engaging with.

Feedback & Review

Student Feedback

After completing the first draft of the assignments, we recruited students to join our focus groups to get feedback on the assignments we created. We held 6 sections of the focus group, with 14 students who took software design. We conducted 30 minute sessions on Monday, Wednesday, and Friday, for two weeks. For each session, students were asked to use various AI

chatbots to assist them with their assignment. On Monday's we focused on using ChatGPT for the assignments, on Wednesdays we focused on Gemini, and on Fridays we focused on Claude. Each student who participated was randomly assigned to one of the 5 assignments in an attempt to have each assignment completed by 3 students. Students had 15 to 20 minutes to complete the assignment followed by another 10 minutes to complete the feedback form. The feedback form included general questions about the use of AI in the assignments, such as "What was your overall experience using AI for this assignment?" "Did you find the use of AI useful?" (Figure A), "Did you feel you learned more, less, or about the same compared to assignments without AI?" (Figure B) and "Would you want AI usage to be expanded, reduced, or kept the same for future assignments" (Figure C), In addition, we asked students specific questions about the assignment they completed.

Students who completed the database assignment were asked, "How did you use AI to fix issues in the database" and "How did AI affect your understanding of databases" to get a better understanding of how students used AI and its effect on completing the database assignment. Similarly, for the other assignments and the extensions of the in-class activities, we asked students questions about how they used AI for that specific assignment and to submit their conversations with the AI chatbots, so we could get a better understanding of how students are using AI for these assignments and to what extent they are useful. Some of the questions we asked students in the form were inspired from the questions in the reflection section of each assignment before it was revised. This reflection process allows students to think critically about the code generated by AI and reflect on the process of each assignment.

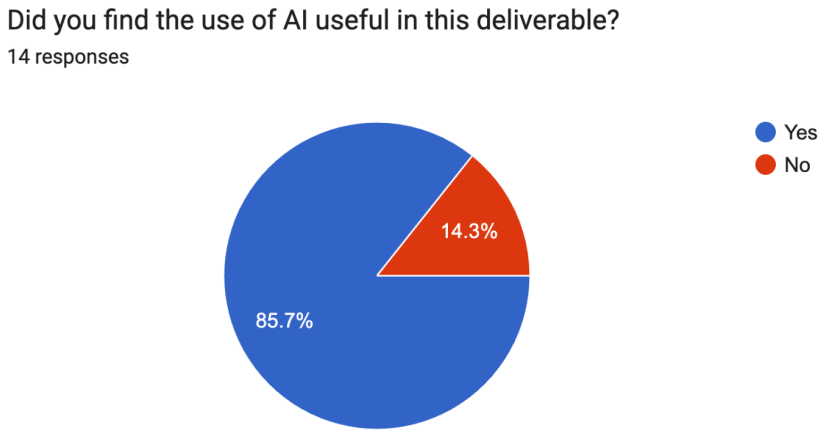

Figure A. Pie graph displaying the results from the feedback form after asking students if they found AI useful in this deliverable.

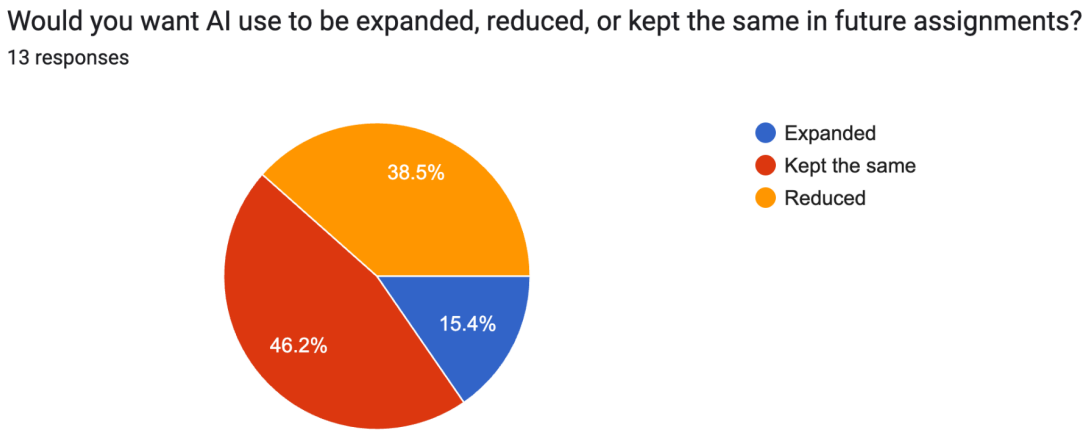

Figure B. Pie graph displaying the results from asking students if they want AI usage to change, specifically expand, reduce, or remain unchanged for future assignments.

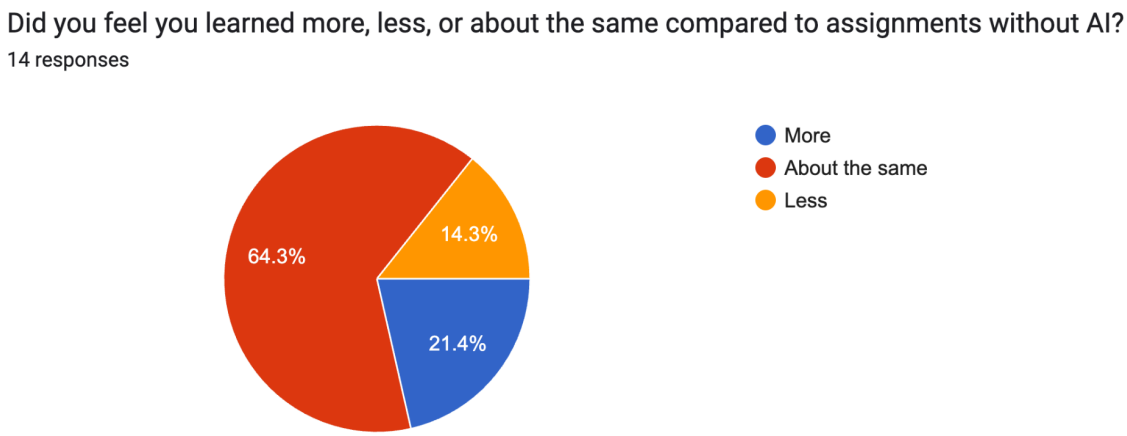

Figure C. Pie graphs showing the results from asking students if they feel like they learned more, less, or about the same compared to assignments they've completed without the use of AI.

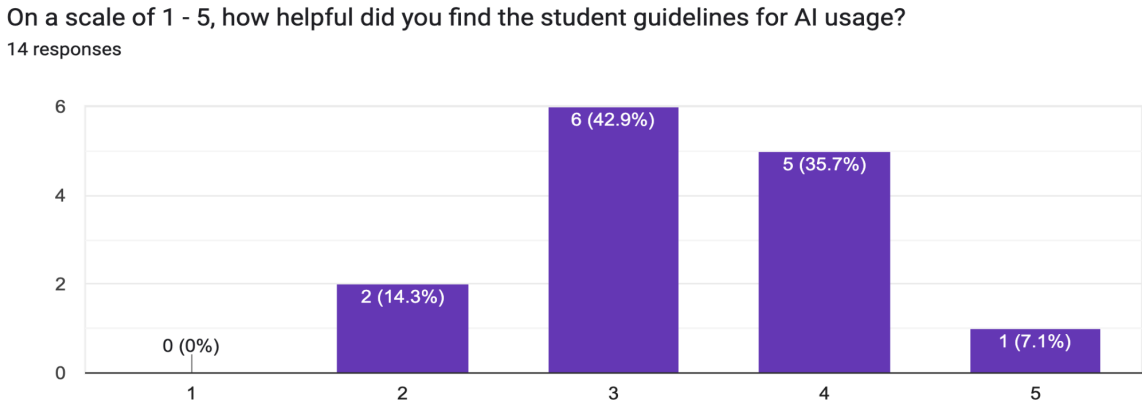

Figure D. Displaying the results of how helpful participants found the student guidelines we created on a scale of 1-5. In this graph, 5 indicates super helpful, 1 indicates not helpful at all.

The feedback students provided led to significant changes within the assignments. Some students stated that they were "struggling to ask the right questions" to the chatbot while others struggled to figure out "how specific to be when prompting" the AI chatbot. On a scale of 1-5, students found the AI guidelines helpful, averaging out at 3 (Figure D), but some students found that the "AI guidelines were too strict compared to the expectation and usage of AI." With this feedback, we made changes to the student guidelines, including more concrete examples which can be referenced for each assignment. For example, in the 'Testing" section of the Student Guidelines, a sample prompt, "what are boundary cases for this function" was provided to give students an idea of the type of appropriate questions and prompting ideas are recommended for the assignments. Within each assignment, students are guided to specific sections of the student guidelines that are directly related to that assignment. For example, in the CSS assignment, we encouraged students to reference the programming section of the student guidelines. In addition to the student guidelines, we provided more guidance within each assignment by providing students with examples of specific questions they could ask AI to provide more guidance in prompting the AI chatbots.

Even after guiding students to specified sections, we still encourage them to read the whole student guidelines page beforehand but we also acknowledge that having more concrete examples can help get a better understanding of the type of prompting needed to complete the assignment. Finally, to make the assignments cohesive, we clarified the details and wording for all assignments. All the assignments now follow the same structure, separating each step with a section header and having consistent information across the front page. We hope that with these changes, students see the consistency between the student guidelines page and the assignments as well as the importance of intentional prompting while using AI in assignments.

Other than struggling with prompting the AI tools, students also found that it was “never explicitly stated not to copy any code” from the chatbots. They thought it would be okay to just copy and paste their output from the chatbot into the assignment and utilize the output. For the html assignment, students are specifically asked to prompt and analyze the results given by the AI chatbot directly. Students need to copy and paste the AI generated code into the preferred IDE to run and examine the results. But for the database and tests assignments, the AI tool should be treated as a learning partner. So, in the database and css sections, we included text that specifically states "Do not blindly accept AI output, treat it as a debugging partner,” in hopes that students understand how to interpret and work with the output provided by the AI chatbot, not directly copy the output.

Professor Feedback

After receiving feedback from students, we inquired professors teaching CS257: Software Design for feedback on the developed assignments. We provided professors a folder including our assignments with all required starter code, a summary of all the created assignments, followed by questions we would like professors to think about when reviewing assignments. to provide feedback on what they thought about the assignments. Due to time constraints, we only received feedback from one professor and our comps advisor.

One major feedback provided highlighted the inconsistencies within our folder and assignment file structures. Initially, we had separated some starter code into separate folders while others stayed within the respective assignments. We changed this structure instead so all of our start code resides in a github repository where students and professors can easily access when necessary. Since our initial idea was for students to complete about 2 of AI integrated assignments, students would only have to clone this repository once to access the required materials, making this process easy and efficient. Every professor has their own preferred method of storing files, but given the context of this class, and skills students are required to learn, Github was the most appropriate place to store the files. When we initially sent the assignment files to professors, the document structure was inconsistent. In one document, we had a tab for backend assignments (for the database and test assignment) and one tab for frontend assignments where the css assignment was located; The html assignment was in its own tab. To keep the files consistent and easy to read, we decided to create a new tab in one document for each assignment, keeping the consistency of all headers, adding appropriate naming conventions, explicitly and clearly stating all instructions.

Diving deeper into the changes we made to individual assignments, for the database assignment. One professor thought it would be a good idea for “students to reflect on the differences between their manual observations and issues in the code they found with the AI.” So we included a question regarding this to get students to think about the differences in what the AI notices versus the student themselves. Within the assignment, we added more obvious and easier errors for students to catch, specifically asking students to identify mismatching of datatypes. We also added an error that may be more difficult for a novice learning student in Software Design

may not detect but an AI chatbot could easily detect. This allows students to think critically and notice the differences between their own observations and the AI's results through that reflection process. For the html assignment, professor feedback gave light to the idea of combining the two types of prompting (general versus iterative) into one part in the assignment, allowing students to practice both types of prompting and reflecting on both processes. Students now have the opportunity to use their html pages in the individual deliverable for this assignment instead of creating a new prompt. This allows students to compare the results of what they expect their html page to look like (referencing their own html page) to the AI generated html page. This comparison is helpful as it allows students to analyze the differences in code generated by AI diving into the complexity from code written by a human.

Finally, for all assignments, insights from one professor led to changes to the questions within the reflection section of each assignment. This section was changed to include more specific questions about each assignment, but also gauge the student to reflect more by including more open-ended questions instead of yes-no questions. We wanted students to engage more and critically think about the code the AI chatbots generated instead of accepting the output as is. Some questions we added were "What are some things the AI found that you wouldn't have found on your own", "Did the AI ever give you incorrect or confusing advice, how did you catch it?", and "What are some things the AI tried to do without you asking?" With these questions, students are able to reflect on their own understanding of the AI generated code and think about where AI actually helped versus hindered in the assignment.

Overall, the student feedback was insightful for all assignments. While we implemented changes based on constructive feedback, students and professors still found the use of AI well integrated and useful for all the assignments. Even though we had a small number of participants in our focus group, we still received profound feedback leading to many changes within our assignments and students guidelines. We hope that in the future, when this assignment is integrated into the CS257:Software Design course, professors will receive more feedback on the usage of AI in these specific assignments but AI usage and AI literacy in the overall course.

Conclusion

The rapid growth of generative AI is responsible for a salient pedagogical issue in the context of teaching and learning about software design. Industry demand for AI proficiency is seemingly at odds with the struggle necessary for "deep learning" and essential skill acquisition. Fully banning AI or giving students free rein leaves those same students either unprepared for the modern, AI-driven workforce or making them all too willing to offload their duties and face a degradation in skills and the quality of the code they produce.

To help mend this pedagogical crisis in CS257 specifically, this project aims to show that AI integration into the course should look more like a delineated collaboration with AI models rather than as an extreme. By establishing clear guidelines rooted in literature about design, programming, testing, and learning, this project defines proper boundaries for students to

observe when doing work. The assignments and exercises created, work to shift the role of generative away from being a code generator to a partner for research, brainstorming, and debugging.

For faculty teaching software design who wish to integrate AI in their curriculum, we have identified several adaptation strategies from testing AI models and holding student focus groups.

- **Context Training:** Instead of having students start generic chats, teaching them to contextualize AI models by providing specifications and starter code helps reduce AI hallucinations and ensures that any assistance rendered by the AI falls within the territory of the course
- **Learning Model Differentiation:** Students tend to gravitate towards different AI tools, so faculty should recognize that different AI models have different pedagogical effects. For example, while some models generate correct but complex code, others produce less polished yet simpler code that's easier to read and digest. Model choice can affect the acquisition of skills.
- **Require Conscious Prompting and Reflections:** In our focus group studies, we found that students often struggle to effectively prompt the models and often choose to simply copy the output code, we've found that structuring the use of ai is beneficial. Particularly, Having students reflect on their use of the AI and what they learned from prompting it helps prevent them from becoming over-reliant.

In summary, AI integration in the cultivation of software development skills should be about the development of AI literacy while cultivating the students' eye for code quality, security, and design. This way, educators can prevent students from stunting the development of their critical thinking and essential skills while ensuring that they have a successful transition to an industry that highly values AI literacy.

Citations

Marie Salomon, Kyle D. Chin, Reid Holmes, Thomas Fritz, and Gail C. Murphy. 2025. An Exploration of How Generative AI Affects Workflow and Collaboration in a Software Engineering Course. In Proceedings of the 2025 ACM SIGPLAN International Symposium on SPLASH-E (SPLASH-E '25). Association for Computing Machinery, New York, NY, USA, 42–54. https://doi.org/10.1145/3758317.3759680

Eric York. 2023. Evaluating ChatGPT: Generative AI in UX Design and Web Development Pedagogy. In Proceedings of the 41st ACM International Conference on Design of Communication (SIGDOC '23). Association for Computing Machinery, New York, NY, USA, 197–201. https://doi.org/10.1145/3615335.3623035

Abdulhalim Musa Abubakar. 2025. Artificial Intelligence Applications in Engineering: A Focus on Software Development and Beyond. *Doupe Journal of Top Trending Technologies* 1, 1 (Jan 2025), 1–11. https://doi.org/10.71063/DJTTT.2025.1101.

Mariana Coutinho, Lorena Marques, Anderson Santos, Marcio Dahia, Cesar França, and Ronnie de Souza Santos. 2024. The Role of Generative AI in Software Development Productivity: A Pilot Case Study. In *Proceedings of the 1st ACM International Conference on AI-Powered Software (AIware '24)*. ACM, New York, NY, USA, 131–138. https://doi.org/10.1145/3664646.3664773.

Rohith Pudari and Neil A. Ernst. 2023. From Copilot to Pilot: Towards AI Supported Software Development. *arXiv preprint arXiv:2303.04142* (2023).